Automata, Grammers and Languages

12.1 ◆ Sequential Circuits and Finite-State Machines

Operations within a digital computer are carried out at discrete intervals of time. Output depends on the state of the system as well as on the input. We will assume that the state of the system changes only at time t = 0, 1,... . A simple way to introduce sequencing in circuits is to introduce a unit time delay.

Definition 12.1.1

A unit time delay accepts as input a bit xt at time t and outputs xt−1, the bit received as input at time t − 1. The unit time delay is drawn as shown in Figure 12.1.1.

Figure 12.1.1 Unit time delay.

As an example of the use of the unit time delay, we discuss the serial adder.

Definition 12.1.2

A serial adder accepts as input two binary numbers

$$x = 0x_N x_{N-1} \cdots x_0 \text{ and } y = 0y_N y_{N-1} \cdots y_0$$

and outputs the sum $z_{N+1}z_N \cdots z_0$ of x and y. The numbers x and y are input sequentially in pairs, $x_0, y_0; \ldots; x_N, y_N; 0, 0$. The sum is output $z_0, z_1, \ldots, z_{N+1}$.

Example 12.1.3

Serial-Adder Circuit

A circuit, using a unit time delay, that implements a serial adder is shown in Figure 12.1.2. Show how the serial adder computes the sum of x = 010 and y = 011.

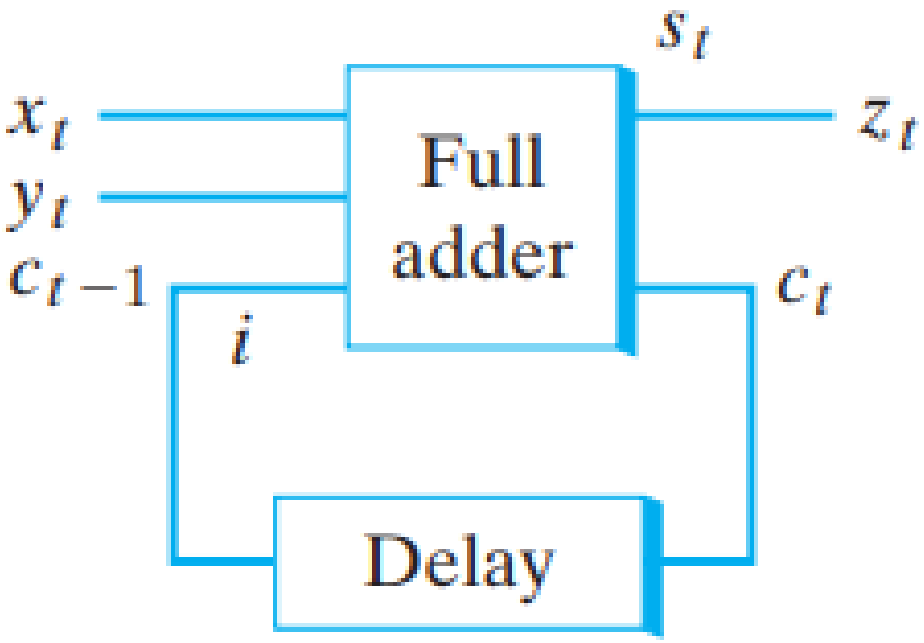

Figure 12.1.2 A serial-adder circuit.

SOLUTION We begin by setting x_0 = 0 and y_0 = 1. (We assume that at this instant i = 0.

This can be arranged by first setting x = y = 0.) The state of the system is shown in Figure 12.1.3(a). Next, we set $x_1 = y_1 = 1$. The unit time delay sends i = 0 as the third bit to the full adder. The state of the system is shown in Figure 12.1.3(b). Finally, we set $x_2 = y_2 = 0$. This time the unit time delay sends i = 1 as the third bit to the full adder. The state of the system is shown in Figure 12.1.3(c). We obtain the sum z = 101.

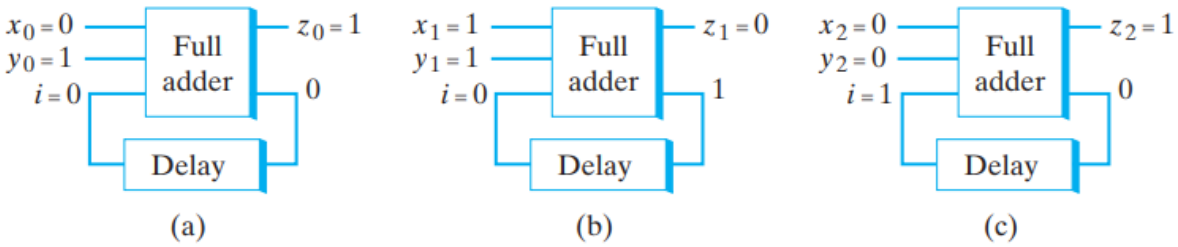

Figure 12.1.3 Computing 010 + 011 with the serial-adder circuit.

A finite-state machine is an abstract model of a machine with a primitive internal memory.

Definition 12.1.4

A finite-state machine M consists of

(a) A finite set $\mathcal{I}$ of *input symbols*.
(b) A finite set $\mathcal{O}$ of *output symbols*.
(c) A finite set $\mathcal{S}$ of *states*.
(d) A *next-state function* f from $\mathcal{S} \times \mathcal{I}$ into $\mathcal{S}$.
(e) An *output function* g from $\mathcal{S} \times \mathcal{I}$ into $\mathcal{O}$.
(f) An *initial state* $\sigma \in \mathcal{S}$.

We write $M = (\mathcal{I}, \mathcal{O}, \mathcal{S}, f, g, \sigma)$.

Example 12.1.5

Let $\mathcal{I} = \{a, b\}$, $\mathcal{O} = \{0, 1\}$, and $\mathcal{S} = \{\sigma_0, \sigma_1\}$. Define the pair of functions $f: \mathcal{S} \times \mathcal{I} \to \mathcal{S}$ and $g: \mathcal{S} \times \mathcal{I} \to \mathcal{O}$ by the rules given in Table 12.1.1.

TABLE 12.1.1

	f		g	
$\mathcal{S}$ \ $\mathcal{I}$	a	b	a	b
σ_0	σ_0	σ_1	0	1
σ_1	σ_1	σ_1	1	0

Then $M = (\mathcal{I}, \mathcal{O}, \mathcal{S}, f, g, \sigma_0)$ is a finite-state machine.

Table 12.1.1 is interpreted to mean

$$
\begin{aligned}
f(\sigma_0, a) &= \sigma_0 \qquad g(\sigma_0, a) = 0,\\
f(\sigma_0, b) &= \sigma_1 \qquad g(\sigma_0, b) = 1,\\
f(\sigma_1, a) &= \sigma_1 \qquad g(\sigma_1, a) = 1,\\
f(\sigma_1, b) &= \sigma_1 \qquad g(\sigma_1, b) = 0.
\end{aligned}
$$

The next-state and output functions can also be defined by a transition diagram. Before formally defining a transition diagram, we will illustrate how a transition diagram is constructed.

Example 12.1.6

Draw the transition diagram for the finite-state machine of Example 12.1.5.

Solution:

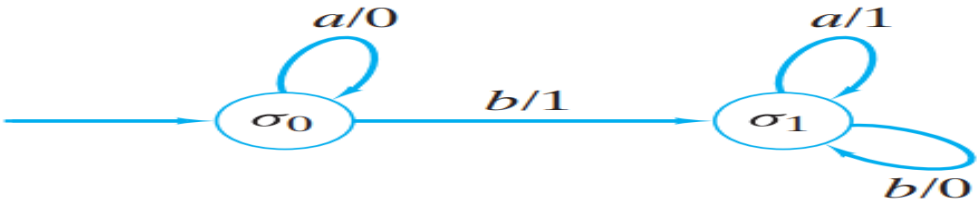

Figure 12.1.4 A transition diagram.

Definition 12.1.8 ▶ Let $M = (\mathcal{I}, \mathcal{O}, \mathcal{S}, f, g, \sigma)$ be a finite-state machine. An *input string* for M is a string over $\mathcal{I}$. The string

$$y_1 \cdots y_n$$

is the *output string* for M corresponding to the input string

$$\alpha = x_1 \cdots x_n$$

if there exist states $\sigma_0, \ldots, \sigma_n \in \mathcal{S}$ with

$$\begin{aligned} \sigma_0 &= \sigma \\ \sigma_i &= f(\sigma_{i-1}, x_i) \qquad \text{for } i = 1, \ldots, n; \\ y_i &= g(\sigma_{i-1}, x_i) \qquad \text{for } i = 1, \ldots, n. \end{aligned}$$

◀

Example 12.1.9

Find the output string corresponding to the input string aababba (12.1.1).

Solution: 0011001.

Example 12.1.10

A Serial-Adder Finite-State Machine

Design a finite-state machine that performs serial addition.

SOLUTION We will represent the finite-state machine by its transition diagram.

Since the serial adder accepts pairs of bits, the input set is {00, 01, 10, 11} and the output set is {0, 1}. Given an input xy, we take one of two actions: Either we add x and y, or we add x, y, and 1, depending on whether the carry bit was 0 or 1. Thus there are two states, which we will call C (carry) and NC (no carry). The initial state is NC. At this point, we can draw the vertices and designate the initial state in our transition diagram (see Figure 12.1.5).

Figure 12.1.5 Two states for the serial-adder finite-state machine.

Next, we consider the possible inputs at each vertex. For example, if 00 is input to NC, we should output 0 and remain in state NC. Thus, NC has a loop labeled 00/0. As another example, if 11 is input to C, we compute 1 + 1 + 1 = 11. In this case we output 1 and remain in state C. Thus C has a loop labeled 11/1. As a final example, if we are in state NC and 11 is input, we should output 0 and move to state C. By considering all possibilities, we arrive at the transition diagram of Figure 12.1.6.

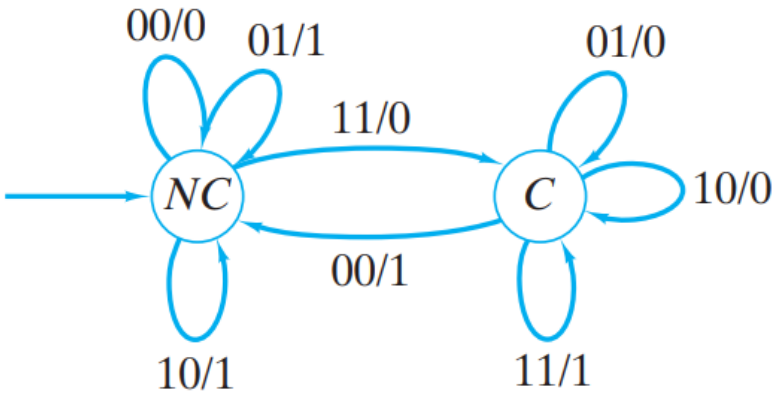

Figure 12.1.6 A finite-state machine that performs serial addition.

12.2 ◆ Finite-State Automata

A finite-state automaton is a special kind of finite-state machine. Finite-state automata are of special interest because of their relationship to languages.

Definition 12.2.1

A finite-state automaton A = (I, O, S, f, g,σ) is a finitestate machine in which the set of output symbols is {0, 1} and where the current state determines the last output. Those states for which the last output was 1 are called accepting states.

Example 12.2.2

Draw the transition diagram of the finite-state machine A defined by the table. The initial state is σ0. Show that A is a finite-state automaton, and determines the set of accepting states.

	f		g	
$\mathcal{S}$ \ $\mathcal{I}$	a	b	a	b
σ_0	σ_1	σ_0	1	0
σ_1	σ_2	σ_0	1	0
σ_2	σ_2	σ_0	1	0

SOLUTION:

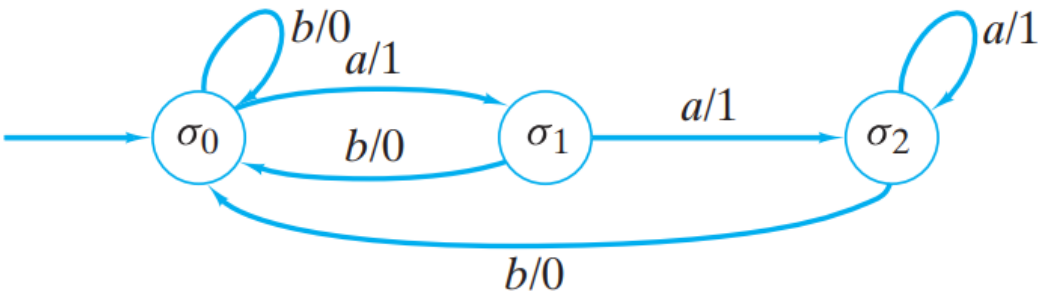

Figure 12.2.1 The transition diagram for Example 12.2.2.

The transition diagram of a finite-state automaton is usually drawn with the accepting states in double circles and the output symbols omitted. When the transition diagram of Figure 12.2.1 is

redrawn in this way, we obtain the transition diagram of Figure 12.2.2.

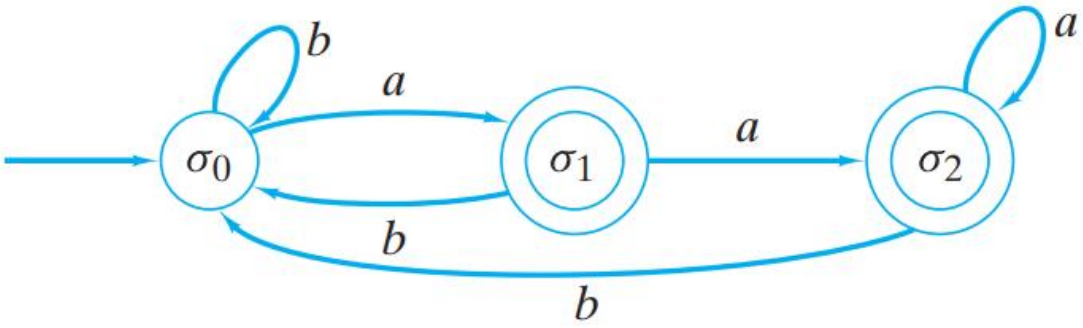

Figure 12.2.2 The transition diagram of Figure 12.2.1 redrawn with accepting states in double circles and output symbols omitted.

Example 12.2.3

Draw the transition diagram of the finite-state automaton of Figure 12.2.3 as a transition diagram of a finite-state machine.

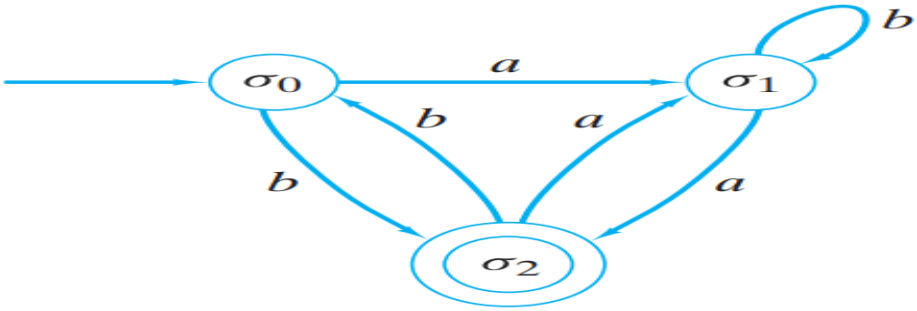

Figure 12.2.3 A finite-state automaton.

Solution:

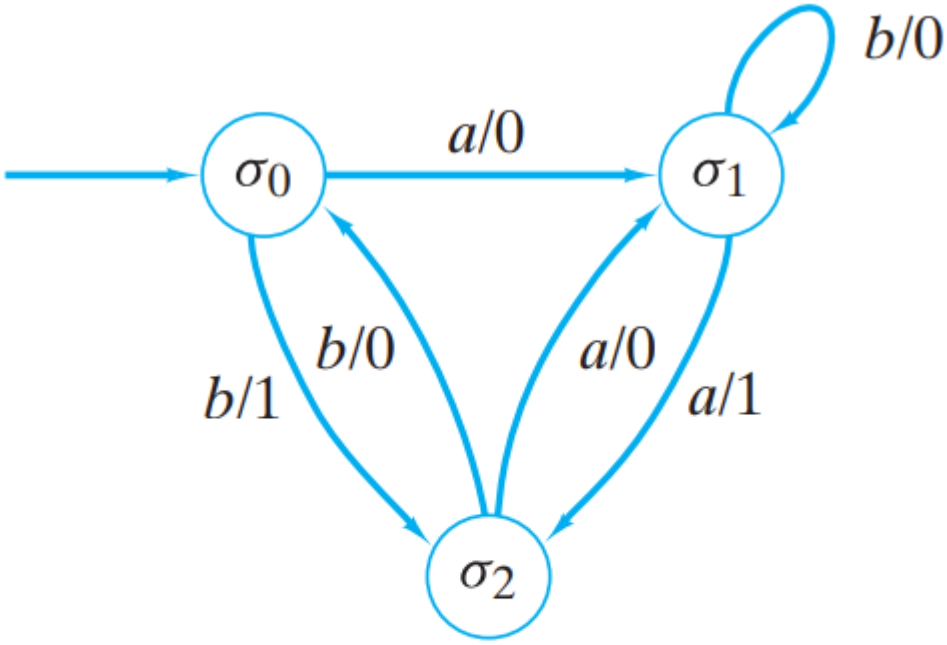

Figure 12.2.4 The finite-state automaton of Figure 12.2.3 redrawn as a transition diagram of a finite-state machine.

As an alternative to Definition 12.2.1, we can regard a finite-state automaton A as consisting of

1. A finite set $\mathcal{I}$ of *input symbols*
2. A finite set $\mathcal{S}$ of *states*
3. A *next-state function* f from $\mathcal{S} \times \mathcal{I}$ into $\mathcal{S}$
4. A subset $\mathcal{A}$ of $\mathcal{S}$ of *accepting states*
5. An *initial state* $\sigma \in \mathcal{S}$.

Example 12.2.4

The transition diagram of the finite-state automaton $A = (\mathcal{I}, \mathcal{S}, f, \mathcal{A}, \sigma)$, where

$$\mathcal{I} = \{a, b\}, \quad \mathcal{S} = \{\sigma_0, \sigma_1, \sigma_2\}, \quad \mathcal{A} = \{\sigma_2\}, \quad \sigma = \sigma_0,$$

and f is given by the following table

	f	
$\mathcal{S}$ \ $\mathcal{I}$	a	b
σ_0	σ_0	σ_1
σ_1	σ_0	σ_2
σ_2	σ_0	σ_2

is shown in Figure 12.2.5.

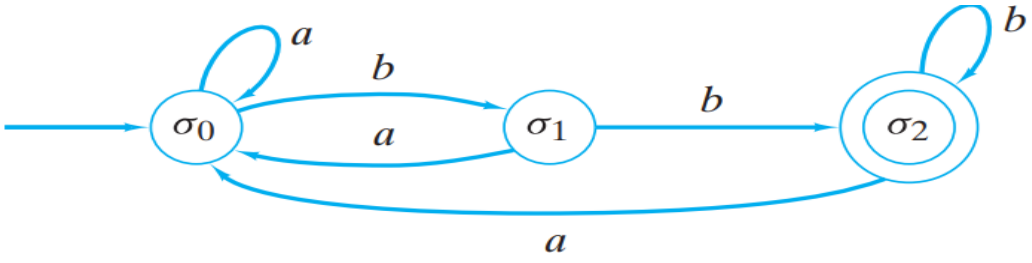

Figure 12.2.5 The transition diagram for Example 12.2.4.

If a string is input to a finite-state automaton, we will end at either an accepting or a nonaccepting state. The status of this final state determines whether the string is accepted by the finite-state automaton.

Definition 12.2.5

Let A = (I, S, f, A,σ) be a finite-state automaton. Let α = $x_1 \cdots x_n$ be a string over I. If there exist states $\sigma_0,\ldots,\sigma_n$ satisfying

(a) σ_0 = σ

(b) f(σ_{i-1}, x_i) = σ_i for i = 1,..., n

(c) $\sigma_n \in$ A,

we say that α is accepted by A.

Let α = $x_1 \cdots x_n$ be a string over I. Define states $\sigma_0,\ldots,\sigma_n$ by conditions (a) and (b)

above. We call the (directed) path $(\sigma_0,\ldots,\sigma_n)$ the path representing α in A.

Example 12.2.6

Is the string abaa accepted by the finite-state automaton of Figure 12.2.2?

Solution: We begin at state σ0. When a is input, we move to state σ1. When b is input, we move to state σ0. When a is input, we move to state σ1. Finally, when the last symbol a is input, we move to state σ2. The path (σ0, σ1, σ0, σ1, σ2) represents the string abaa. Since the final state σ2 is an accepting state, the string abaa is accepted by the finite-state automaton of Figure 12.2.2.